

Микросервисная система цифрового обслуживания сети кафе с ИИ-рекомендациями и учетом макронутриентного состава блюд

Выполнил: Пономаренко Никита Михайлович

Ментор: Семченко Александр Андреевич, Руководитель команды разработки операционной платформы шеринговых сервисов, Яндекс

Дата защиты: 29.06.2026

- Сфера общественного питания активно цифровизируется.
- Выбор блюда – сложная задача: разнообразие позиций меню, состав, макронутриенты, аллергены, предпочтения
- Возможность консультации сотрудника кафе зависит от загруженности персонала и плохо масштабируется

Поэтому актуальна разработка приложения, которое объединяет цифровое меню, профиль пользователя, фильтрацию блюд и систему рекомендаций

Обзор существующих решений

Starbucks + ChatGPT

- + Подбор напитков по настроению, вкусу и контексту
- Зависимость от внешней платформы (все происходит через интерфейс ChatGPT)

Wendy's FreshAI

- + Автоматизация приема заказа
- + Снижение нагрузки на персонал
- Фокус на оформлении заказа, не на рекомендации варианта

Цифровые меню

- + Фильтрация по аллергенам, нутриентам, диетическим ограничениям
- Нет рекомендаций по выбору, только ручной отбор

Общая проблема: фильтрация, персонализированные рекомендации и оформление заказа не существуют как объединенные функции.

Приложение nutrio объединяет эти части в одном backend-контуре.

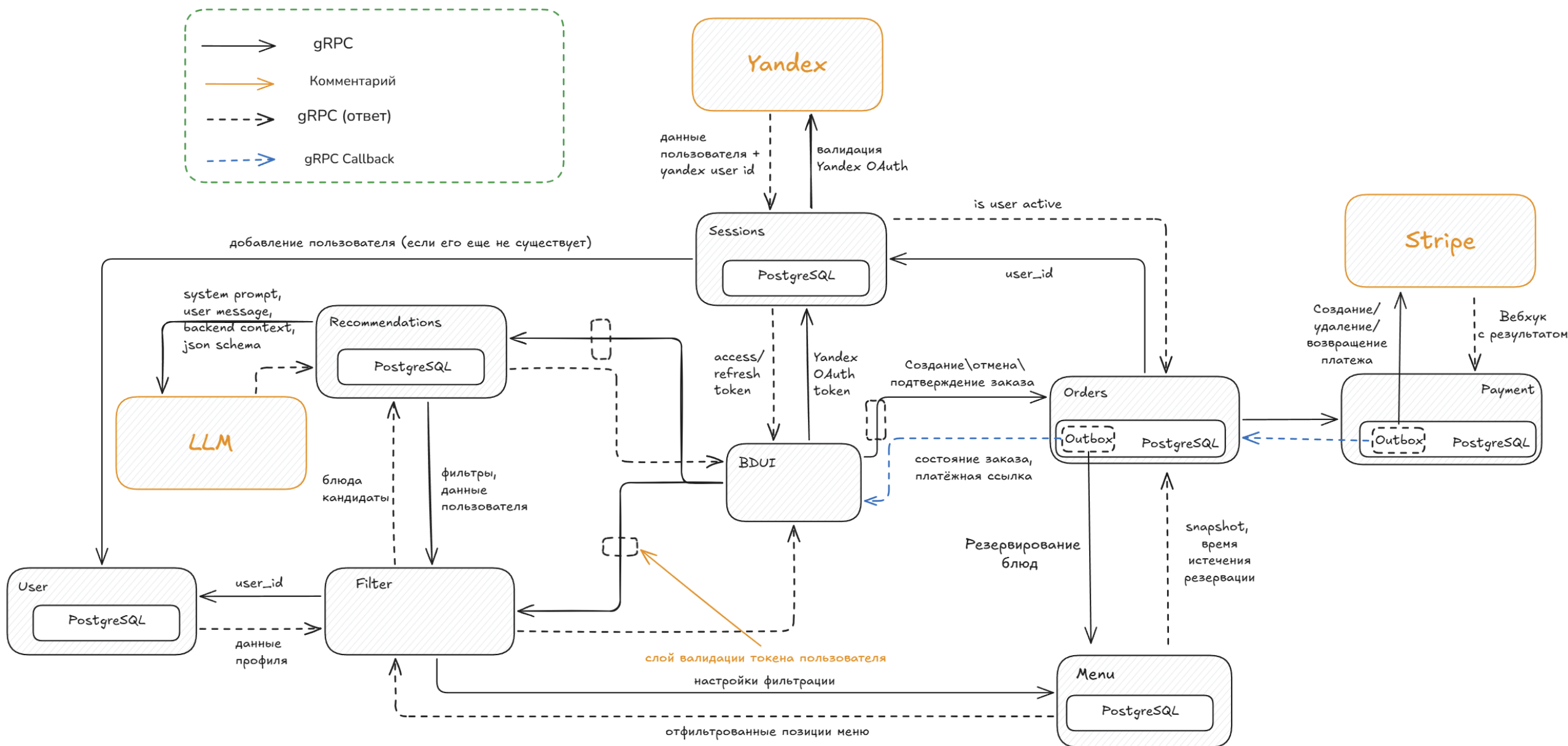
Цель в рамках моей зоны ответственности

Спроектировать и реализовать backend-сервисы авторизации, фильтрации меню и AI-рекомендаций, обеспечив надежное межсервисное взаимодействие, централизованную обработку пользовательской авторизации, профильных ограничений и параметров подбора блюд.

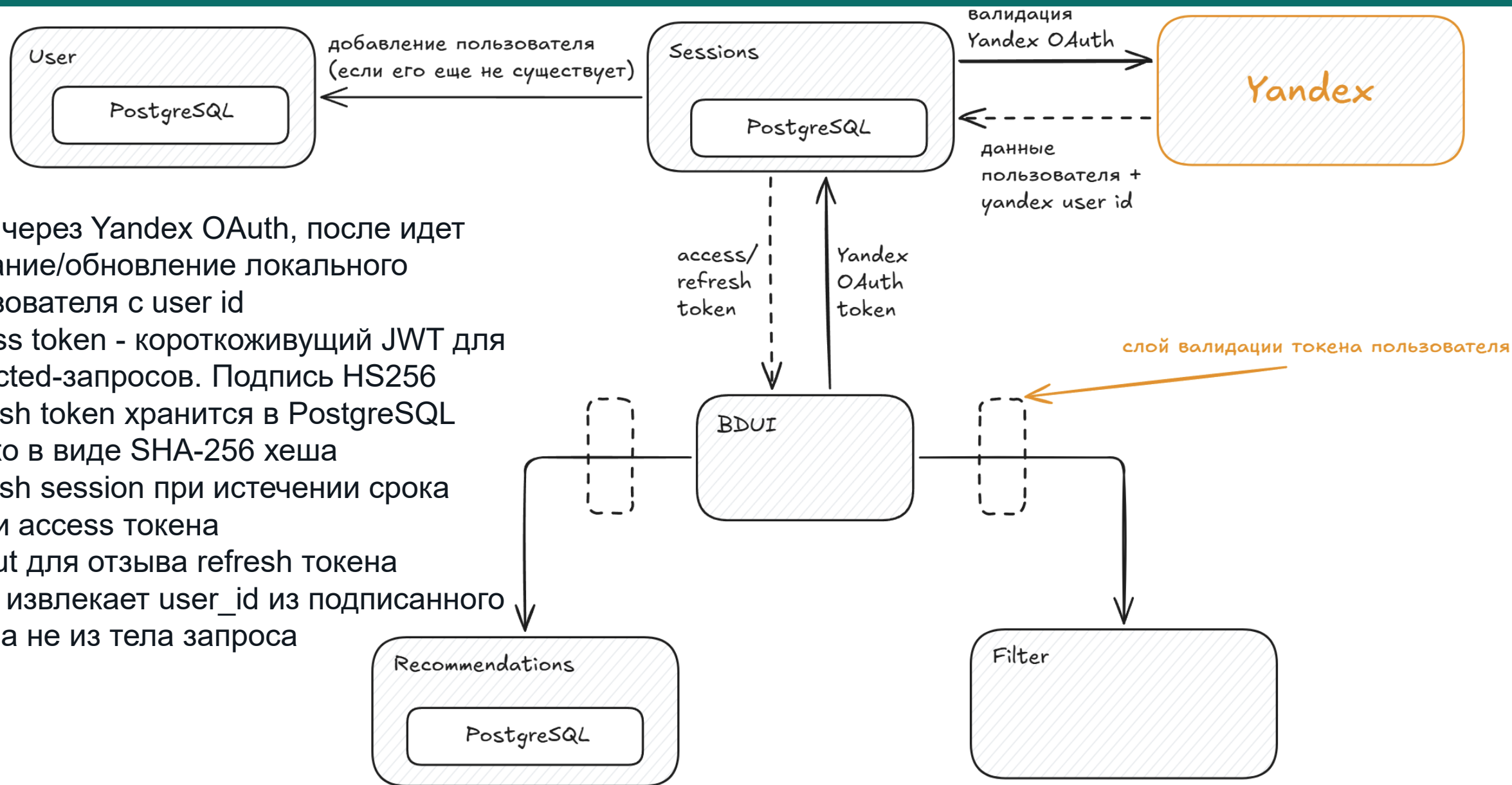
Задачи

- Спроектировать и реализовать Sessions service для авторизации пользователей через Yandex OAuth, выпуска JWT и управления refresh-сессиями.
- Спроектировать и реализовать Filter service для построения единого effective filter на основе параметров запроса и ограничений пользовательского профиля.
- Спроектировать и реализовать Recommendations service для асинхронного выбора наиболее подходящего блюда из списка, полученного по пользовательским фильтрам.
- Реализовать межсервисные gRPC-контракты и интеграции между Sessions, Filter, Menu, Recommendations и BDUI gateway.
- Интегрировать Recommendations service с LLM, обеспечив строгую JSON-схему ответа, валидацию результата, повторную обработку невалидных ответов и доставку рекомендации клиенту через event-систему.

Общая архитектура и моя зона ответственности

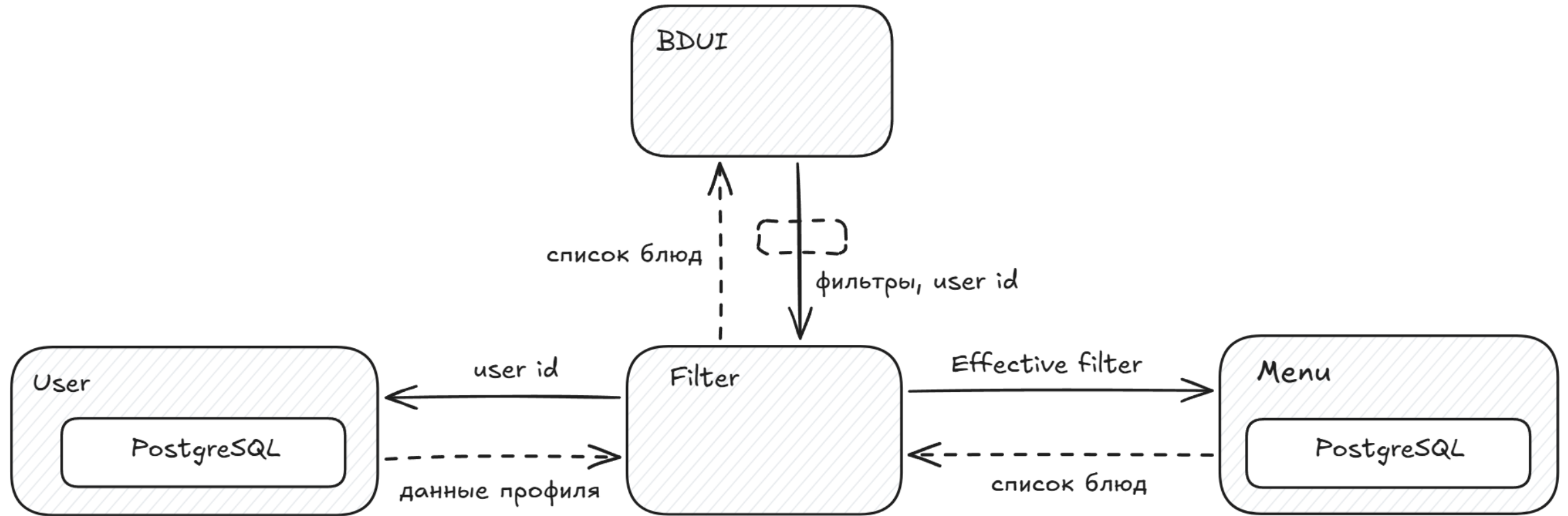


Sessions service



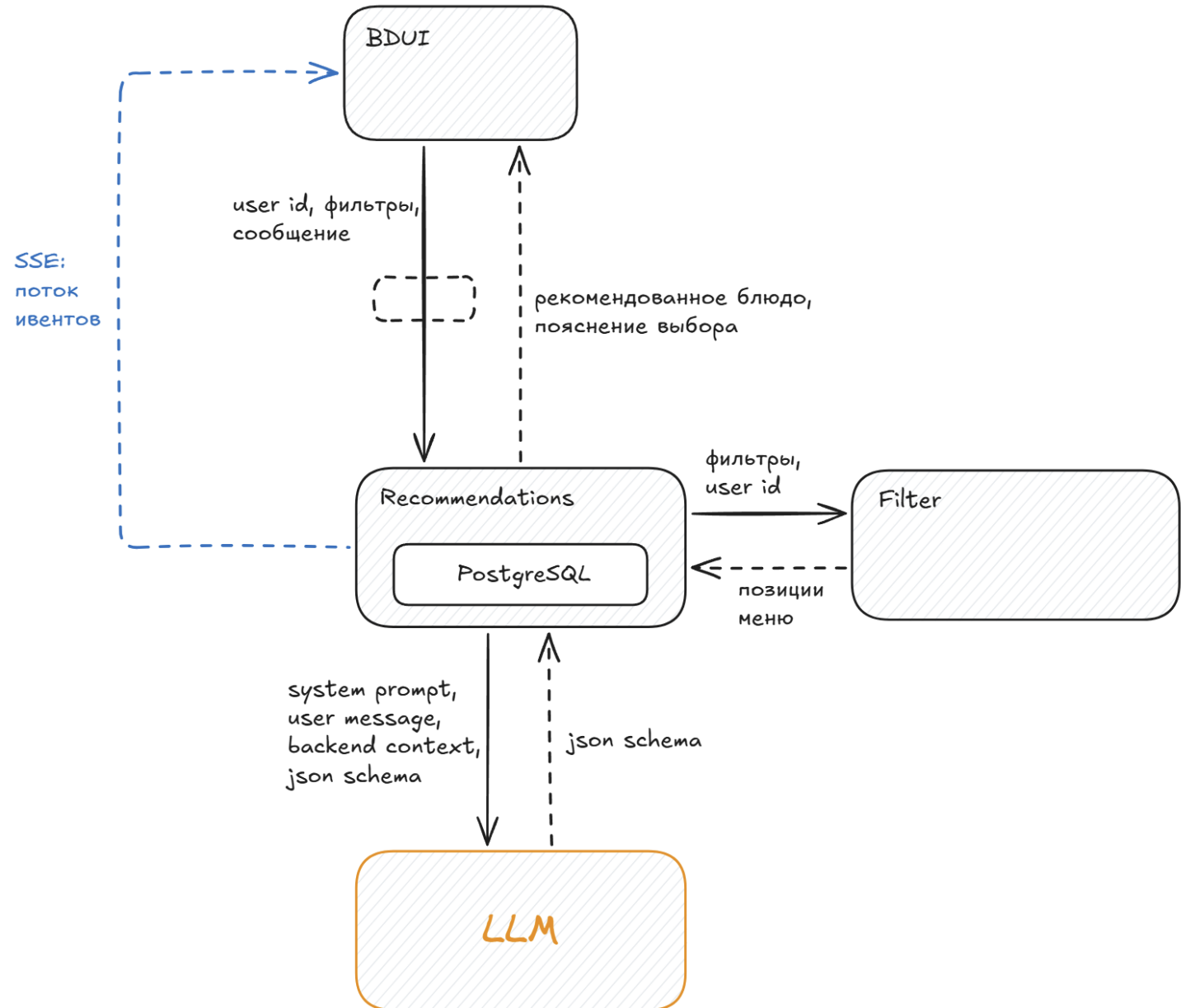
- Вход через Yandex OAuth, после идет создание/обновление локального пользователя с user id
- Access token - короткоживущий JWT для protected-запросов. Подпись HS256
- Refresh token хранится в PostgreSQL только в виде SHA-256 хеша
- Refresh session при истечении срока жизни access токена
- Logout для отзыва refresh токена
- BUI извлекает user_id из подписанного JWT, а не из тела запроса

Filter service



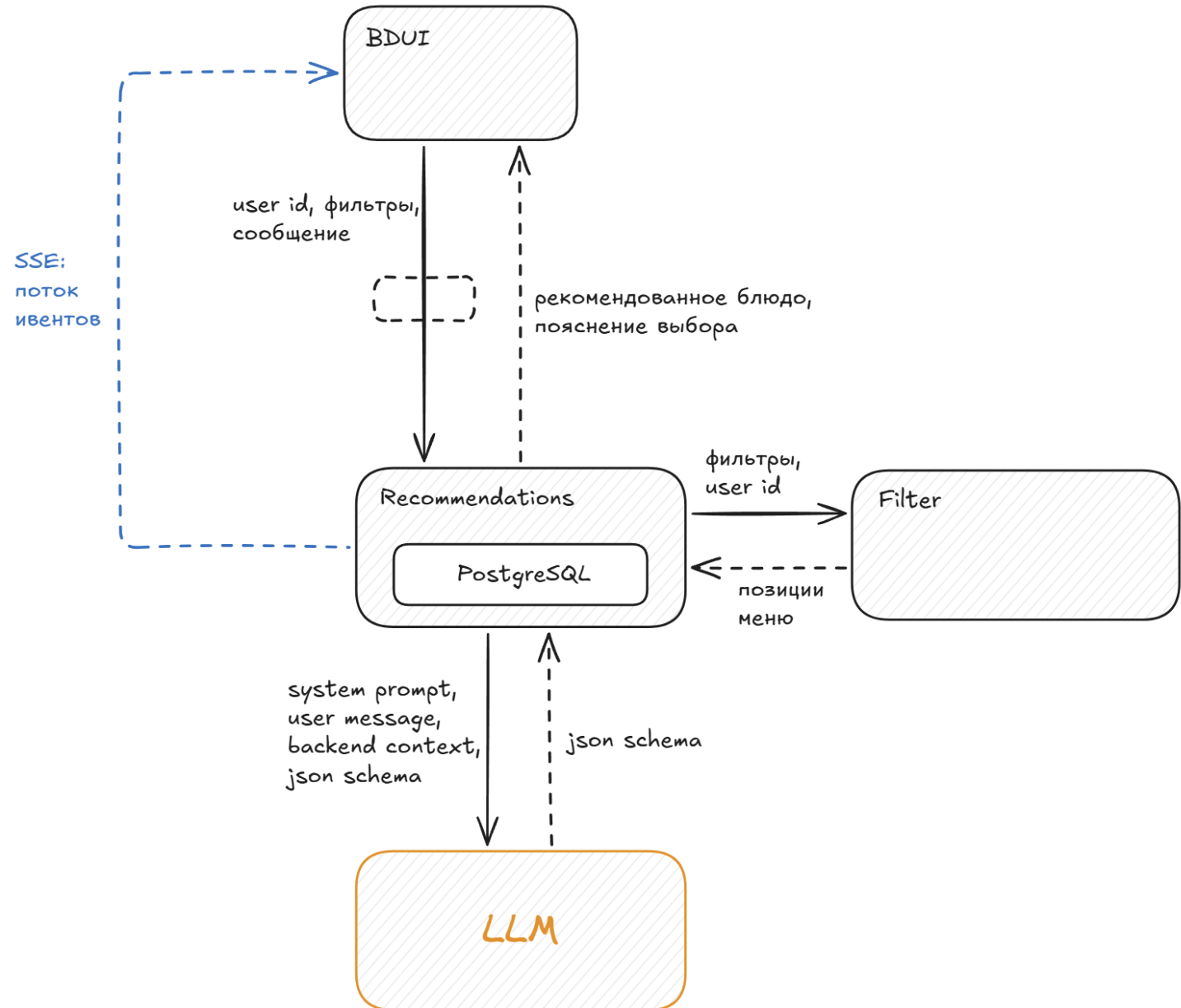
Recommendations service

- Принцип: база данных — источник истины, LLM ничего не придумывает, формирует выбор, основываясь только на переданном контексте
- Системный промпт: полное описание задачи и ограничений для LLM.
- Строгая json_schema ответа LLM: assistant message + 1-5 причин выбора.
- Асинхронная job-worker система
- Job сохраняется в PostgreSQL. Имеет начальный статус 'queued', переходный статус 'processing', финальные статус 'completed', 'failed'
- Worker pollит 'queued' jobs. Переводит в статус в 'processing', создает события



Recommendations service

- Worker выполняет запрос в filter, проходит LLM pipeline
- Worker валидирует ответ LLM
- При невалидных ответах – repair-loop
- SSE для получения событий изменения статусов job
- События хранятся в PostgreSQL
- Реализована возможность реконнекта пользователя: получение пропущенных ивентов по запросу
- Сохранение аудита запросов к LLM для дебага, анализа затрат по времени, частоте ошибок, токенам на запросы



Sessions

Smoke/auth-flow:

- mock Yandex OAuth
- access/refresh token
- Refresh
- Logout
- protected BDUI routes

Filter

Unit tests:

- построение EffectiveFilter
- объединение запроса и профиля
- избранное и ограничения

Recommendations

Unit/scenario tests:

- Создание job
- Worker flow
- Парсинг JSON-ответа
- Validation layer
- Repair-loop

Сравнение моделей Yandex: от YandexGPT 5 Pro до Alice-AI-LLM-Flash. Для текущего one-shot сценария достаточно Alice-AI-LLM-Flash.

Достигнутые результаты

- Реализован Sessions service: вход через Yandex OAuth, создание/обновление локального пользователя, выпуск JWT access token и refresh token, хранение refresh token в виде хеша, refresh и logout.
- Реализован слой JWT-валидации в BDUI gateway: защищенные ручки получают `user\_id` из подписанного access token, а не из тела запроса клиента.
- Реализован Filter service: построение Effective Filter: объединение пользовательских фильтров с профильными ограничениями и получение подходящих блюд через Menu service.
- Реализован Recommendations service: асинхронная recommendation job, worker-обработка, получение кандидатов через Filter service, LLM-выбор одного блюда, JSON-схема ответа, validation layer, repair-loop и аудит вызовов LLM.
- Реализована event-доставка результата: события сохраняются в PostgreSQL, доставляются через gRPC stream и преобразуются в SSE для мобильного клиента.
- Ключевые части Filter и Recommendations покрыты unit-тестами, а пользовательские сценарии рекомендаций проверены smoke/сценарными тестами.
- В составе командного результата сервисы интегрированы в приложение Nutrio: авторизация, фильтрация меню, учет ограничений, AI-рекомендация

Командный результат: полноценное интернет-меню для сети кафе, предоставляющая функционал по фильтрации, рекомендации с учетом профильных настроек, оформлении заказа.

- Диалоговый сценарий вместо one-shot
- Checkpoint/action-модель под контролем backend
- Более богатый LLM-контекст: алиасы, синонимы, вкусовые признаки
- Метрики качества рекомендаций
- Тесты SSE/reconnect и repair-loop

Спасибо за внимание!